Hack the Highway — handoff plan

For: a teammate's AI coding assistant (and the teammate). Repo: **github.com/DurgeshTAMU/hack-the-highway-control-tower** (private, branch **main**). Updated 2026-11-07, evening. Read this whole document before changing code.

0. How to use this document

- Sections 1–3 are **context you must not re-derive**: the challenge, the architecture, and facts already verified against the data. Treat them as established.
- Section 4 is **what is done**, with evidence. Section 5 is **what is left**, in priority order, each with acceptance criteria. Section 6 lists **rules for working on this repo**. Section 7 is a command sheet.
- Nothing here is a fabricated result. Numbers come from the recorded run `runs/run4` (rule reader, current code). Anything not yet measured is marked **TBD**.

1. The challenge in brief

Build an operations control tower for Wacky Wheels Co., a fictional just-in-time automaker: one coordinating agent plus three specialist agents. It reads 15 free-text events (`data/events.json`), uses an LLM to understand each one, builds and traverses a knowledge graph over the company tables, and executes corrective actions through a local mock server (`enterprise_server.py`), following `data/policy_handbook.md`. Everything runs on a laptop; any model may be used.

Scoring component	Weight	What it means for us
Event handling accuracy	50%	The submitted action_log.jsonl is replayed on a fresh world and compared with the expected outcome. Exact orders moved, dealers/customers told, loaners, lots, vehicles; noise left alone; rumors escalated; extra actions penalized.
Knowledge graph and reasoning	20%	Ontology quality, correct traversal, a graph path cited per event.
Agent architecture and guardrails	10%	Clear roles, LLM-versus-code split, validation before acting, failed-call handling, escalation.
Efficiency	10%	Few wasted or failed calls; small or cheap models. The model used must be cited.
UX and explainability	10%	Control-tower view: event feed, graph path, actions, audit trail, per-event summary.

Submission: action_log.jsonl from a run on the 15 events, source code, ontology export or diagram, a trace per event, a demo video or screenshots, a one-page design card, and the model(s) cited. **Rules:** no hard-coded answers, must work from event text and tables; escalate anything not specific or confirmed.

2. Architecture: Ground, Compile, Prove

The core idea, which must be preserved: **the LLM reads; the knowledge graph decides which reading is real; the handbook is compiled into an exact plan by code; the plan is proven on a local copy of the world before any live call.** Code locations are in `control_tower/`.

Stage	Module	What it does
Read	nlp.py, llm.py	Rule reader (lexical) and LLM reader (JSON schema). The LLM returns only the incident kind, surface names and raw date phrases; code maps names to ids (aliases.csv plus table names) and phrases to dates. Each relative date keeps all candidate meanings.
Ground	ground.py	Expands each reading into hypotheses (candidate ids × date conventions) and keeps only those that bind to real records: a shipment matching supplier/part/plant/ETA, a lot matching the build window, a plant plus outage window. One binding: act. None or conflicting: escalate. No operational entity: noise. Hedged wording: escalate. The date convention is chosen per message so that every incident grounds.

Stage	Module	What it does
Graph	kg.py, impact.py	networkx MultiDiGraph: 11 classes, 17 relations, about 17k nodes and 28k edges. Inferred edge Lot -INSTALLED_IN-> ProductionOrder (plant, window, BOM). Impact sets are traversals that record the edges walked; those edges are the cited graph path.
Compile	agents.py	Coordinator; SupplyChainPlanner (one shared slot ledger per event; earliest feasible slot; transfer only when the home plant is full); DealerNetwork (one notify per dealer listing only its affected orders; stop_sale); CustomerCareQuality (loaner priority Platinum, Gold, then promised date, then id, capped by pool; hold_lot; notify owners).
Prove	verify.py, clients.py	Dry run on the unmodified enterprise.py in-process (TwinClient), then 13 invariants checked against an oracle that recomputes expectations from raw CSV rows, not the graph. The plan is withheld and escalated if any check fails.
Execute	pipeline.py, clients.py	HTTP client; a failed move is re-read with mes.get_schedule and retried once; otherwise escalate.
Explain	pipeline.py, app.py, replay.py	Per-event trace JSON (readings, hypotheses, verdict, path, plan, proof, calls, audit, summary); Streamlit UI; judge-style replay of a log.

3. Facts already verified (do not re-derive)

- **Today is Mon 2027-03-08** (working-day index 0); the planning window runs to Fri 2027-04-02. Every plant has 5 slots a day and exactly 3 future orders a day, so only **2 slots a day are free**. Capacity binds.
- **“next <weekday>” means that weekday in the following week.** This is proven by E-01: Quanta’s chip shipment to Princeton exists only with ETA Mar 15, and “can't make it next Monday… next Friday now” grounds only if next Friday = Mar 19. The same message’s outage is therefore Mar 19–22 (6 orders), not Mar 12–22 (21 orders).
- Name traps: “Lumen” is only in suppliers.csv (SUP-18), not aliases.csv; “Volt” is a harness supplier (SUP-05), not a battery; two suppliers make CHIP-IVI, so the supplier must match the lot.
- The BOM filter decides precision. E-14 affects 180 of 268 vehicles built in its window (Terra uses HARN-C); E-15 affects 116 of 185 (Ranger has no CHIP-IVI). E-09 and E-10 share plant and dates but affect different models.
- crm.notify_customer is for quality holds only. Delayed customers are reached only through dealer.notify.
- Read tools (mes.get_schedule, crm.loaner_availability) and escalate are not writes and are not penalized.

Per-event result of the recorded run (runs/run4, rule reader, transfer day = original date):

Event	Category	Grounded to	Impact	Writes
E-01	Outage + supplier delay	P_TX Mar 19–22; SH-0190 CHIP-IVI P_IN Mar 15→19	6 + 8 orders moved	81
E-02	Unconfirmed	— (vague “seat material”, no dates)	1 escalate	0
E-03	Quality hold	LOT-0130 (Kestrel CHIP-IVI P_MS Feb 25–Mar 1)	260 vehicles	188
E-04	Plant outage	P_KY Mar 16–18 (convention default)	9 orders	59
E-05–07	Noise	— (dealer brochure questions)	no calls	0
E-08	Plant outage	P_KY Mar 24–25	6 orders	46
E-09	Supplier delay	SH-0256 CAM-360 P_AL Mar 29→Apr 1	6 orders (2 transfers)	42
E-10	Supplier delay	SH-0248 BATT-60 P_AL Mar 29→Apr 1	3 orders	28
E-11	Plant outage	P_MS Mar 26–30	9 orders (3 transfers)	62
E-12	Supplier delay	SH-0182 CAM-360 P_IN Mar 15→18	3 orders	27
E-13	Unconfirmed	— (vague “something electronic”)	1 escalate	0
E-14	Quality hold	LOT-0017 (Atlas HARN-B P_TX Feb 22–24)	180 vehicles	131
E-15	Quality hold	LOT-0191 (Quanta CHIP-IVI P_IN Mar 2–3)	116 vehicles	102

Totals: 879 live calls, 0 failed, 2 escalations (rumors only), 0 writes on noise or rumors. Replay of the log on a fresh world: all 15 events pass every invariant, and the independent audit (tools/audit_run.py) finds every handbook rule holding.

4. What is done

Item	Status and evidence
Knowledge graph and ontology export	Done. runs/run4/ontology/ontology.json and ontology.mmd; knowledge_graph.graphml is regenerated per run (git-ignored, 7.5 MB).
Rule reader and grounding	Done. Handles nicknames, relative dates, vague part words resolved by grounding, unnumbered compounds, open-start outages (“down through Wed” = today..Wed).
LLM reader	Done. Teammate ran qwen2.5:1.5b in Ollama on a Mac (runs/final, 15 LLM calls). **runs/final is not in the repo yet.**
Four agents and the policy compiler	Done. Shared slot and loaner ledgers for compound events; one notice per dealer.
Proof before action	Done. 13 invariants, table-only oracle, plan withheld if any fails.
Failed-call handling	Done. Re-read and retry once for moves; live pool check for loaners; transport retries with backoff; escalate otherwise.
Live run and submission log	runs/run4 (rule reader, current code) is in the repo. The LLM run runs/final must be re-run on the current code and pushed.
Policy decisions	Decided: transfer day = original date; scarce slots by original date then id; E-04 'next Tuesday' = Mar 16.
Independent audit	tools/audit_run.py: imports none of our code, replays the log into enterprise.py, checks every rule against a hand-read answer key. ALL RULES HOLD; it caught 4 of 4 corrupted logs.
Tool discovery (MCP-style)	Catalog read from GET /tools; every call checked (name, argument names, types) before sending; malformed calls blocked and escalated.
Replay checker	Done: python -m control_tower.replay runs/run4 prints ALL OK.
Tests	31 passing, incl. tool-catalog blocking, the narrowed LLM-misread rule, plain-language escalations, and the independent audit on the full feed.
Control-tower UI	Done and reviewed in a browser. Tabs: Control tower (graph path drawn with real relations, actions with handbook-rule tags and sample messages, loaner queues, slot heatmap, proof, how dates were read), Human review inbox, Live replay (for the demo), Efficiency & model, Ontology, Audit log & downloads (action log, submission zip).
Reports and ontology image	tools/export_reports.py writes one Markdown report per event plus ontology.svg.
Design card and README	Updated. The model is cited (qwen2.5:1.5b).

5. What is left, in priority order

Pri	Task	Details	Done when
P0	Re-run the LLM submission on the current code and push it	On the machine with Ollama: pull main, start a fresh server into runs/final, run with CT_LLM=ollama and CT_LLM_MODEL=qwen2.5:1.5b. The LLM-misread rule was narrowed (an LLM reading that names a plant, part or supplier the grounded incident does not cover now escalates), so confirm escalations are still exactly 2 (E-02, E-13). .gitignore already allows runs/final/.	Replay ALL OK; python -I tools/audit_run.py runs/final prints ALL RULES HOLD; no trace contains llm_error; runs/final pushed; DESIGN_CARD.md numbers match its run_summary.json.
P1	Record the demo video	Use the UI's Live replay tab for events arriving, then show a delay (E-09: graph path, loaners, schedule), a quality hold (E-14), a rumor in Human review, and Efficiency & model. 2-3 minutes.	Video file in the submission folder.

Pri	Task	Details	Done when
P1	Package the submission	Audit log & downloads tab, then 'Submission bundle (.zip)' on runs/final, plus source code, DESIGN_CARD.md and the video.	Matches the submission list in section 1.
P2	Robustness extras	More unseen-wording tests; consider whether far-away news mentioning “shipments” should be noise rather than escalation (currently escalated, which is safe but may be scored as the wrong category); optional LLM-written dealer messages behind a validator.	Tests stay green, the audit still holds, and the 15-event results are unchanged.

6. Rules for working on this repo

- **Do not hard-code answers** (event ids, expected orders, lots). Everything must come from event text and the tables. Tests may name events; library code may not.
- **Do not modify enterprise.py, enterprise_server.py or data/.** They are the organisers' systems. The twin imports enterprise.py unmodified.
- **Keep the split: the LLM reads, code computes.** Never let the model emit ids, ISO dates or actions directly into a tool call.
- **After any change:** run python -m pytest tests -q, then an offline run, and confirm the per-event table in section 3 is unchanged unless the change was intended. Before submitting, do a live run plus replay.
- **The server starts once per run** and refuses to overwrite an existing log; use a new --out folder, or --overwrite deliberately.
- **Never commit tokens or keys.** Use environment variables; .gitignore already excludes .env and *.token.
- Windows note: TwinClient shares one Enterprise instance per process; world ids are made unique by a process-wide counter. Do not reuse a fixed event id across dry runs.
- Commit small, with clear messages. Do not rewrite main's history.

Key files: control_tower/ (code), tests/test_control_tower.py, DESIGN_CARD.md, CONTROL_TOWER_README.md, tools/ (audit, report export), runs/run4/ (current rule-reader run).

7. Command sheet (PowerShell)

```
pip install networkx streamlit altair pandas pytest
python -m pytest tests -q                                    # expect 31 passed

# offline, no server (executes on the local twin)
python -m control_tower.run --twin --out runs/offline

# live: terminal 1
python enterprise_server.py --data data --events data/events.json --out runs/final --port 8000
# live: terminal 2
python -m control_tower.run --server http://127.0.0.1:8000 --out runs/final
python -m control_tower.replay runs/final                    # expect ALL OK
python -I tools/audit_run.py runs/final                      # expect ALL RULES HOLD
python tools/export_reports.py runs/final                    # per-event reports + ontology.svg

# UI
streamlit run control_tower/app.py

# LLM via Ollama (local), set in the terminal that runs the agents
ollama pull qwen2.5:1.5b
$env:CT_LLM = "ollama"; $env:CT_LLM_MODEL = "qwen2.5:1.5b"
# LLM via an OpenAI-compatible API (e.g. GitHub Models)
$env:CT_LLM = "openai"; $env:CT_LLM_URL = "https://models.github.ai/inference"
$env:CT_LLM_MODEL = "microsoft/Phi-4-mini-instruct"; $env:CT_LLM_API_KEY = "<token with models:read>"
```